

## READING SHEET

# Forms & Controlled Components

Module 3 · Frontend: React & Next.js · Day 33 · Week 7

### PROGRAM

CodeOps · Full Stack

### WORK TYPE

Revision + mini-project

### EST. TIME

2–3 hours

### Tip How to use this sheet

This is your take-away for Day 33. The checkout is where Addis Eats either takes an order or loses one, so this sheet treats a form as an interface rather than a set of inputs: the mechanics first, then validation rules and their timing, then feedback everyone can perceive, and finally React Hook Form. Build the form by hand before you reach for the library — you need to know what it is doing on your behalf.

## Part 1 · Form Fundamentals

Every field lives in one state object, and one handler updates whichever field fired. Give each input a name attribute and use a computed key — adding a fifth field then costs one line of JSX and nothing else.

```

Checkout.jsx – the whole form in one state

const [form, setForm] = useState({
  name: "", phone: "", area: "Bole", notes: ""
});

function handleChange(e) {
  const { name, value } = e.target;
  setForm((f) => ({ ...f, [name]: value }));
}



```

Use the updater form — `setForm(f => ...)` — so you read the latest state rather than the value captured at render. In a form where several fields change quickly, that difference matters.

### Binding each input type

| INPUT            | WHAT YOU BIND                                            |
|------------------|----------------------------------------------------------|
| text, tel, email | value + onChange, read <code>e.target.value</code>       |
| textarea         | value + onChange — not children as in HTML               |
| select           | value on the select, not selected on the option          |
| checkbox         | checked + onChange, read <code>e.target.checked</code>   |
| file             | Uncontrolled — read <code>e.target.files</code> yourself |

Two exceptions are worth memorising: a checkbox uses `checked` rather than `value`, and a file input cannot be controlled at all — the browser will not let you set its value.

### Submitting properly

```
submit once, and say so

async function handleSubmit(e) {
  e.preventDefault();      // no page reload
  if (submitting) return;   // no double order
  setSubmitting(true);
  try {
    const order = await placeOrder(form);
    navigate(`/orders/${order.id}`, { replace: true });
  } finally { setSubmitting(false); }
}
```

Put the handler on the **form**, not the button. That way the Enter key works, screen readers announce it correctly, and the browser's own behaviour is respected. On a weak mobile signal a student *will* press Order twice — the submitting guard and a disabled button are what stop two identical orders reaching the kitchen.

### The six states a form can be in

| STATE      | WHAT THE PERSON SHOULD SEE                      |
|------------|-------------------------------------------------|
| pristine   | No errors yet — they have not typed anything    |
| dirty      | Their input, kept exactly as typed              |
| invalid    | What is wrong, beside the field it belongs to   |
| submitting | A disabled button and a clear "Sending..."      |
| failed     | Why it failed, and their data still in the form |
| succeeded  | Confirmation, or a redirect to the receipt      |

#### Warning Never clear the form on failure

Most beginner forms handle only "typing" and "sent". The other four states are where trust is won or lost — particularly failure. Making someone retype a phone number and an address because your server was slow is the fastest way to lose the order.

## Part 2 · Validation

Validation answers two questions: is this value acceptable, and *when* should I say so. The rules are the easy half; the timing is what separates a form people finish from one they abandon.

```
validate.js – a pure function per form

export function validate(form) {
  const errors = {};
  if (!form.name.trim())
    errors.name = "Please enter your name";
  if (!/^(?:\+251|0)9\d{8}$/.test(form.phone))
    errors.phone = "Use 09... or +2519... (TeleBirr number)";
  return errors; // {} means valid
}
```

It takes the form and returns an errors object — no React, no DOM, no side effects, exactly like the reducer on Day 30 and just as easy to test alone. Call it during render so the errors are **derived** and can never disagree with the values.

### When to show an error

| TIMING                      | HOW IT FEELS                                    |
|-----------------------------|-------------------------------------------------|
| On every keystroke          | Scolds you before you have finished typing      |
| On blur                     | Waits until you leave the field — usually right |
| On submit                   | Fine for short forms, frustrating for long ones |
| After first blur, then live | The pattern most good forms use                 |

```
const [touched, setTouched] = useState({});  
  
onBlur={() => setTouched(t => ({ ...t, phone: true })))}  
  
const showPhoneError = touched.phone && errors.phone;
```

track which fields have been visited

An empty field is invalid from the very first render, but the person has not done anything wrong yet. Tracking which fields they have actually visited is what makes the timing humane. Once a field has shown an error, switch to validating on every keystroke so the message disappears the moment they fix it.

### Write messages people can act on

| UNHELPFUL            | ACTIONABLE                                  |
|----------------------|---------------------------------------------|
| Invalid input        | Use 09... or +2519...                       |
| Required             | We need a name for the delivery             |
| Pattern mismatch     | A TeleBirr number has 10 digits             |
| Something went wrong | That number is not registered with TeleBirr |

Name the field, describe the expected shape, and use words the person already knows. Ethiopian mobile numbers arrive as 0911... or +251911... — accept both, strip spaces before testing, and never reject a number for formatting you could have fixed yourself.

#### Warning The server validates too

Client validation is a courtesy: fast feedback so nobody waits for a round trip to learn they missed a field. Server validation is the actual rule, because anything sent from a browser can be forged. Handle a 422 by reading the field errors out of the response and showing them beside the right inputs — you will build that side in Module 4.

## Part 3 · Messaging & Feedback

An error message nobody sees, or that a screen reader never announces, has not done its job. Four attributes turn a plain input into one that explains itself.

Field.jsx – label, input, message

```
<label htmlFor="phone">TeleBirr number</label>

<input id="phone" name="phone"
  value={form.phone} onChange={handleChange}
  aria-invalid={!showError}
  aria-describedby={showError ? "phone-error" : undefined} />

{showError && <p id="phone-error" role="alert">{errors.phone}</p>}
```

htmlFor and id connect the label to the input; aria-invalid marks the field as wrong; aria-describedby ties the message to it; and role="alert" makes a screen reader announce it. A placeholder is not a label — it disappears the moment someone types, and a real label also gives a bigger tap target on a phone.

### Colour is never the only signal

Around one man in twelve has some colour vision deficiency, and a long checkout on a small screen can push the failing field out of sight entirely. Use three signals together: the message in **words** beside the field, a mark or icon that does not depend on colour, and **focus** moved to the first invalid field on a failed submit.

focus the first problem

```
const firstError = Object.keys(errors)[0];
if (firstError) {
  document.getElementById(firstError)?.focus();
  return;
}
```

### Feedback while it is working

the button carries the state

```
<button type="submit" disabled={submitting || hasErrors}>
  {submitting ? "Sending your order..." : "Order – 640 ETB"}
</button>

{serverError && <p role="alert">{serverError}</p>}
```

Putting the total in the button label means nobody confirms an order without seeing what it costs — a small detail that removes the most common reason people abandon a checkout. And only disable the button once the person can see what is stopping them: a disabled button with no visible errors just looks broken.

## Part 4 · React Hook Form

Everything so far you wrote by hand, and you should be able to. React Hook Form is what most professional React projects use instead. Its central choice is to be **uncontrolled**: rather than storing every keystroke in React state, it registers the real DOM inputs and reads them when needed — which on a twenty-field form is the difference between smooth and sluggish.

| BY HAND                       | REACT HOOK FORM              |
|-------------------------------|------------------------------|
| One state object you maintain | The library holds the values |
| Re-render on every keystroke  | Almost no re-renders         |
| Your own touched tracking     | formState.touchedFields      |

| BY HAND                       | REACT HOOK FORM              |
|-------------------------------|------------------------------|
| A validate function you write | Rules declared on each field |
| Manual submitting flag        | formState.isSubmitting       |

Checkout.jsx — the same form, fewer lines

```
const { register, handleSubmit, formState } = useForm({
  defaultValues: { name: "", phone: "", area: "Bole" }
});
const { errors, isSubmitting } = formState;

<form onSubmit={handleSubmit(placeOrder)}>
  <input {...register("name", { required: "Please enter your name" })} />
  <input {...register("phone", { pattern: {
    value: /^(?:\+251|0)9\d{8}$/,
    message: "Use 09... or +2519..." } })} />
</form>
```

register returns name, ref, onChange and onBlur — spreading it onto the input is what hands the field to the library. handleSubmit validates everything and only calls your function when the form is actually valid.

## Schemas

one schema, both sides

```
const schema = z.object({
  name: z.string().min(1, "Please enter your name"),
  phone: z.string().regex(TELEBIRR, "Use 09... or +2519...")
});

useForm({ resolver: zodResolver(schema) });
```

### Tip The library gives you the message, not the design

A zod schema is a plain object describing the shape of valid data, and the same file can validate the form in the browser and the request body on the server — which you will do in Module 4. But where the message appears, how it is announced and where focus goes are still entirely your decisions: everything in Part 3 still applies.

## Choosing an approach

**By hand** for two or three fields with simple rules, or when other parts of the screen need to read the values live. **React Hook Form** for many fields, real validation, or forms large enough that per-keystroke re-renders show. **Add a schema** when the same shape must be checked on the server too.

## Assigned Reading

### Required

1. [react.dev — Reacting to input with state](https://react.dev/learn/reacting-to-input-with-state) What / why: thinking about a form as a set of states rather than a set of fields.
2. [react-hook-form.com — Get started](https://react-hook-form.com/get-started) What / why: register, handleSubmit, formState and declaring validation rules.

### Optional — go deeper

- [GOV.UK — Error messages](#) — how to word and place errors, written from real user research.
- [Zod — basic usage](#) — declaring a data shape once and reusing it on both sides of the wire.

## Exercises

Work these in order and commit as you go.

1. Build the checkout with name, TeleBirr phone, delivery area and optional notes in one state object.
2. Add a select for the area (Bole, Kazanchis, Megenagna, Piassa) and bind it correctly.
3. Write `validate(form)` as a pure function and call it during render.
4. Track touched fields on blur and show each error only after its field has been visited.
5. Label every field and wire `aria-invalid`, `aria-describedby` and `role="alert"`.
6. Add a submitting flag, disable the button, and put the ETB total in its label.
7. Simulate a failed request: show the reason, keep every value, focus the first bad field.

## Mini-Project · The Addis Eats Checkout

### Tip What you build

A complete checkout form that handles all six states honestly. Four fields in one state object, rules in a pure function, errors that appear when they help rather than nag, feedback that works without colour and with a screen reader, and a submit that cannot fire twice or lose the order when the network fails.

## Requirements

- All four fields controlled from one state object with a single change handler.
- A pure `validate(form)` returning an errors object, derived on every render.
- Errors shown only after a field is touched, then updating live as it is corrected.
- A real label per field, plus `aria-invalid`, `aria-describedby` and `role="alert"`.
- A submitting state that disables the button and shows the ETB total in its label.
- A failure path that keeps every value and moves focus to the first invalid field.

## A worked start

```
starter – derived errors and touched
const [form, setForm] = useState({ name: "", phone: "", area: "Bole" });
const [touched, setTouched] = useState({});
const errors = validate(form);
const hasErrors = Object.keys(errors).length > 0;

const show = (field) => touched[field] && errors[field];

<input id="phone" name="phone" value={form.phone}
  onChange={handleChange}
  onBlur={() => setTouched(t => ({ ...t, phone: true })))}
  aria-invalid={!show("phone")} />
```

## What to submit

Your GitHub repository with Checkout.jsx, validate.js and Field.jsx, plus a README note listing each rule and why it exists.

**Check yourself**

- Does pressing Enter in a text field submit the form?
- Can you complete the whole form using only the keyboard?
- Does an untouched empty field stay quiet until you leave it?
- Does a corrected field clear its message immediately?
- Does pressing Order twice quickly send only one order?
- After a simulated failure, is every value still in the form?
- Turn the screen greyscale — can you still tell which field is wrong?